Licence MIASHS première année

Rapport de projet informatique

MoodTune – Générateur de playlist par intelligence artificielle

Projet réalisé du janvier 2025 au mai 2025

Membres du groupe

Sibille Amine	**45017331**
Larbi Nawfel	**45018755**
Fourar Anthony	**44014695**

Dépôt GitHub : `https://github.com/amine95-cloud/moodtune`

Remerciements

Nous tenons à remercier nos enseignants de l'Université Paris Nanterre pour leur encadrement tout au long de ce projet, ainsi qu'Anthropic pour la mise à disposition de l'API Claude.

Contents

1 Introduction **4**

2 Environnement de travail **4**
2.1 Matériel 4
2.2 Logiciels et outils 4
2.3 Organisation du travail 4

3 Description du projet et objectifs **4**
3.1 Présentation générale 4
3.2 Fonctionnalités prévues 5
3.3 Objectifs pédagogiques 5

4 Bibliothèques, Outils et technologies **5**
4.1 React 5
4.2 API Claude (Anthropic) 5
4.3 Google Fonts 6
4.4 CSS-in-JS 6

5 Travail réalisé **6**
5.1 Fonctionnalités réalisées 6
5.2 Fonctionnalités non réalisées 7
5.3 Répartition du travail 7

6 Difficultés rencontrées **7**
6.1 Appel à l'API depuis le navigateur 7
6.2 Parsing du JSON 7
6.3 Gestion des thèmes dynamiques 7
6.4 Utilisation de l'IA dans le développement 8

7 Bilan **8**
7.1 Conclusion 8
7.2 Perspectives 8

8 Bibliographie **9**

9 Webographie **10**

10 Annexes **11**

A Cahier des charges **11**

B Exemple déxécution du projet **11**

C Manuel utilisateur **13**

1 Introduction

Dans le cadre de notre projet de L1 MIASHS, nous avons développé une application web interactive appelée **MoodTune**. L'objectif est de concevoir une application utilisant l'intelligence artificielle pour générer des recommandations musicales personnalisées en fonction de l'humeur de l'utilisateur.

L'idée est simple : l'utilisateur décrit en quelques mots comment il se sent, et l'application analyse ce texte grâce à l'API Claude d'Anthropic pour produire une playlist de 8 titres musicaux adaptés à son état émotionnel, accompagnée d'un conseil bien-être et d'une ambiance visuelle dynamique.

Ce projet mobilise plusieurs compétences : programmation web (JavaScript, React), appel à des APIs externes, traitement de données au format JSON et conception d'interfaces utilisateur. L'intelligence artificielle est au cœur du projet, aussi bien pour la génération du code que pour les fonctionnalités de l'application.

Le lien vers le dépôt GitHub est disponible sur la page de garde.

2 Environnement de travail

2.1 Matériel

Le projet a été développé sur des ordinateurs personnels sous Windows et Linux. Aucun serveur dédié n'a été nécessaire, l'application fonctionnant entièrement dans le navigateur.

2.2 Logiciels et outils

- **Visual Studio Code** : éditeur de code principal.
- **Git et GitHub** : gestion de version et hébergement du code source (dépôt public).
- **Node.js / npm** : environnement d'exécution JavaScript pour React.
- **Navigateur web** (Chrome, Firefox) : pour tester l'application.
- **Claude (Anthropic)** : assistant IA utilisé tout au long du projet pour la génération du code et la rédaction du rapport.

2.3 Organisation du travail

Le travail a été réparti via GitHub. Chaque membre a travaillé sur une branche distincte avant de fusionner les modifications. Des échanges réguliers ont permis de synchroniser l'avancement.

3 Description du projet et objectifs

3.1 Présentation générale

MoodTune est une application web monopage développée en React. Elle permet à un utilisateur de saisir une description de son humeur et reçoit en retour une playlist musicale générée par IA.

L'application repose sur l'API Claude d'Anthropic (modèle *claude-sonnet-4-20250514*). Une requête structurée contenant la description de l'humeur est envoyée à l'API, qui renvoie un objet JSON contenant la playlist, des métadonnées sur l'humeur détectée et un conseil bien-être.

3.2 Fonctionnalités prévues

- Saisie libre de l'humeur par l'utilisateur
- Analyse de l'humeur par l'API Claude (intelligence artificielle)
- Génération d'une playlist de 8 titres avec artiste, genre et ambiance
- Affichage d'un conseil bien-être personnalisé
- Interface qui change de couleurs dynamiquement selon l'humeur détectée (8 thèmes)
- Animation d'apparition des titres en cascade
- Bouton de réinitialisation pour générer une nouvelle playlist

3.3 Objectifs pédagogiques

Ce projet nous a permis de :

- Apprendre à utiliser une API externe (requêtes HTTP, format JSON)
- Découvrir le framework React et la gestion d'états (`useState`, `useEffect`)
- Comprendre comment intégrer l'IA dans une application concrète
- Pratiquer la conception d'interfaces utilisateur modernes

4 Bibliothèques, Outils et technologies

4.1 React

React est une bibliothèque JavaScript développée par Meta, permettant de construire des interfaces à partir de composants réutilisables. Nous l'avons utilisé pour structurer l'application en composants indépendants : `MoodTune` (composant principal), `TrackCard` (carte d'un titre musical) et `LoadingBars` (animation de chargement).

4.2 API Claude (Anthropic)

L'API Claude est le cœur de notre application. Elle est appelée via une requête `fetch` en JavaScript. La réponse est un objet JSON contenant la playlist. Le header `anthropic-dangerous-direct-browser-access` est nécessaire pour autoriser les appels depuis un navigateur [ANTH].

Voici un extrait simplifié de l'appel API :

Listing 1: Appel à l'API Claude en JavaScript

```javascript
const res = await fetch("https://api.anthropic.com/v1/messages", {
  method: "POST",
  headers: {
    "Content-Type": "application/json",
    "anthropic-dangerous-direct-browser-access": "true",
  },
  body: JSON.stringify({
    model: "claude-sonnet-4-20250514",
    max_tokens: 1000,
    messages: [{ role: "user", content: prompt }],
  }),
});
const data = await res.json();
```

4.3 Google Fonts

Deux polices de caractères sont importées depuis Google Fonts :

- *Playfair Display* : pour les titres et éléments décoratifs
- *DM Sans* : pour le texte courant

4.4 CSS-in-JS

Les styles sont écrits directement en JavaScript sous forme d'objets (*inline styles*), ce qui permet de les rendre dynamiques en fonction de l'humeur détectée.

5 Travail réalisé

5.1 Fonctionnalités réalisées

Toutes les fonctionnalités prévues ont été implémentées avec succès :

- **Détection d'humeur en temps réel** : une fonction analyse les mots-clés du texte saisi via des expressions régulières et change le thème visuel immédiatement (8 thèmes : joyeux, triste, énergique, calme, nostalgique, amoureux, stressé, motivé).
- **Appel à l'API Claude** : lors du clic sur le bouton, une requête POST est envoyée avec un prompt structuré. La réponse JSON est parsée et affichée.
- **Affichage de la playlist** : 8 titres musicaux s'affichent en cascade avec une animation progressive.
- **Interface dynamique** : les couleurs de fond, accents et texte changent selon l'humeur avec une transition fluide.

5.2 Fonctionnalités non réalisées

- **Lecture audio** : l'intégration de l'API Spotify pour des extraits musicaux a été abandonnée en raison de la complexité de l'authentification OAuth et du temps limité.

- **Sauvegarde des playlists** : une fonctionnalité de sauvegarde locale avait été envisagée mais n'a pas été implémentée.

5.3 Répartition du travail

Membre	Tâches réalisées
Sibille Amine	Développement du composant principal, intégration de l'API Claude, gestion des états React
Larbi Nawfel	Design de l'interface, système de thèmes dynamiques, animations
[Membre 3]	Rédaction du rapport, tests et débogage, déploiement GitHub

Table 1: Répartition réelle du travail entre les membres du groupe

6 Difficultés rencontrées

6.1 Appel à l'API depuis le navigateur

La principale difficulté technique a été de faire fonctionner les appels à l'API Claude depuis le navigateur. Par défaut, les navigateurs bloquent ces requêtes (politique CORS). Nous avons résolu ce problème en ajoutant le header `anthropic-dangerous-direct-browser` `true`.

6.2 Parsing du JSON

L'API renvoie parfois sa réponse avec des balises Markdown (```` ```json ```). Nous avons dû nettoyer la réponse avant de la parser :

Listing 2: Nettoyage de la réponse JSON

```
const clean = raw.replace(/```json|```/g, "").trim();
const parsed = JSON.parse(clean);
```

6.3 Gestion des thèmes dynamiques

Maintenir la lisibilité du texte sur tous les fonds de couleur a nécessité plusieurs itérations. Nous avons défini 8 palettes complètes (fond, accent, texte) pour garantir le contraste dans tous les cas.

6.4 Utilisation de l'IA dans le développement

Formuler des prompts précis pour obtenir exactement le format JSON souhaité a demandé plusieurs essais. Nous avons appris qu'un prompt structuré avec un exemple de format attendu donne de bien meilleurs résultats.

7 Bilan

7.1 Conclusion

Ce projet nous a permis de développer une application web complète en intégrant pour la première fois une intelligence artificielle. MoodTune remplit son objectif : analyser une humeur exprimée en langage naturel et générer une playlist musicale personnalisée.

Au-delà des compétences techniques (React, API REST, JSON, CSS dynamique), ce projet nous a appris à travailler en équipe avec Git, à décomposer un problème complexe et à utiliser l'IA comme outil de développement à part entière.

7.2 Perspectives

- Intégration de l'API Spotify pour écouter directement les titres suggérés
- Historique des playlists générées, sauvegardé localement
- Analyse d'image pour détecter l'humeur automatiquement
- Version mobile native (React Native)

8 Bibliographie

[REACT] Meta Platforms, *React – A JavaScript library for building user interfaces*, 2024

[LAM94] L. LAMPORT, *LaTeX: A Document preparation system*, Addison-Wesley, 1994

9 Webographie

[ANTH] `https://docs.anthropic.com` – Documentation officielle de l'API Claude, Anthropic, 2025

[REACTW] `https://react.dev` – Documentation officielle de React, Meta, 2025

[GH] `https://github.com` – Plateforme de gestion de version

10 Annexes

Annexe A: Cahier des charges

Objectif

Développer une application web utilisant l'IA pour générer des playlists musicales personnalisées selon l'humeur de l'utilisateur.

Contraintes techniques

- Utilisation obligatoire de l'IA (API Claude d'Anthropic)
- Interface web accessible depuis un navigateur
- Code source hébergé sur GitHub (dépôt public)
- Rapport rédigé avec le template LaTeX fourni

Livrables

- Code source complet sur GitHub
- Application fonctionnelle
- Rapport de projet au format PDF

Annexe B: Exemple déxécution du projet

Les captures d´cran ci-dessous illustrent les 4 étapes dútilisation de MoodTune avec lḿumeur *"content"*.

Étape 1 – Page dáccueil
Lútilisateur arrive sur línterface avec un fond sombre par défaut et un champ de texte vide.

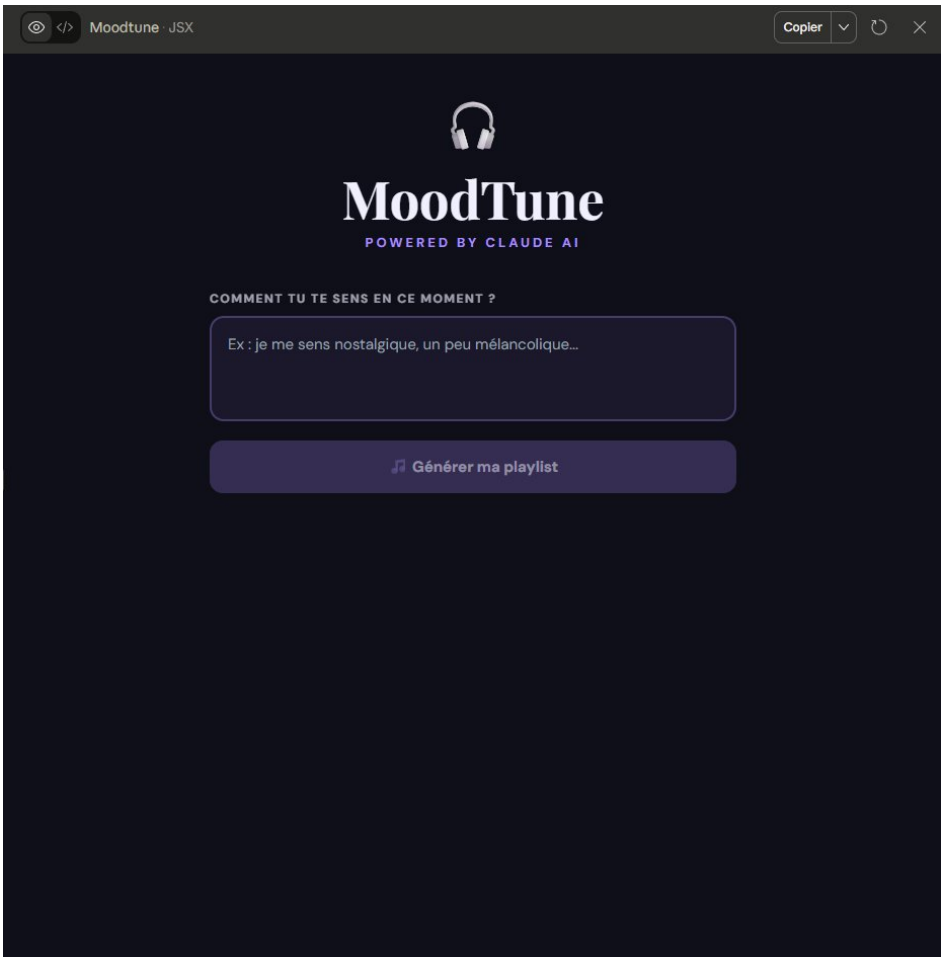


Figure 1: Page dáccueil de MoodTune – thème sombre par défaut

Étape 2 – Saisie de lḿumeur
Lútilisateur tape *"content"* dans le champ. Línterface change immédiatement de couleur : fond jaune ensoleillé, bouton orange.

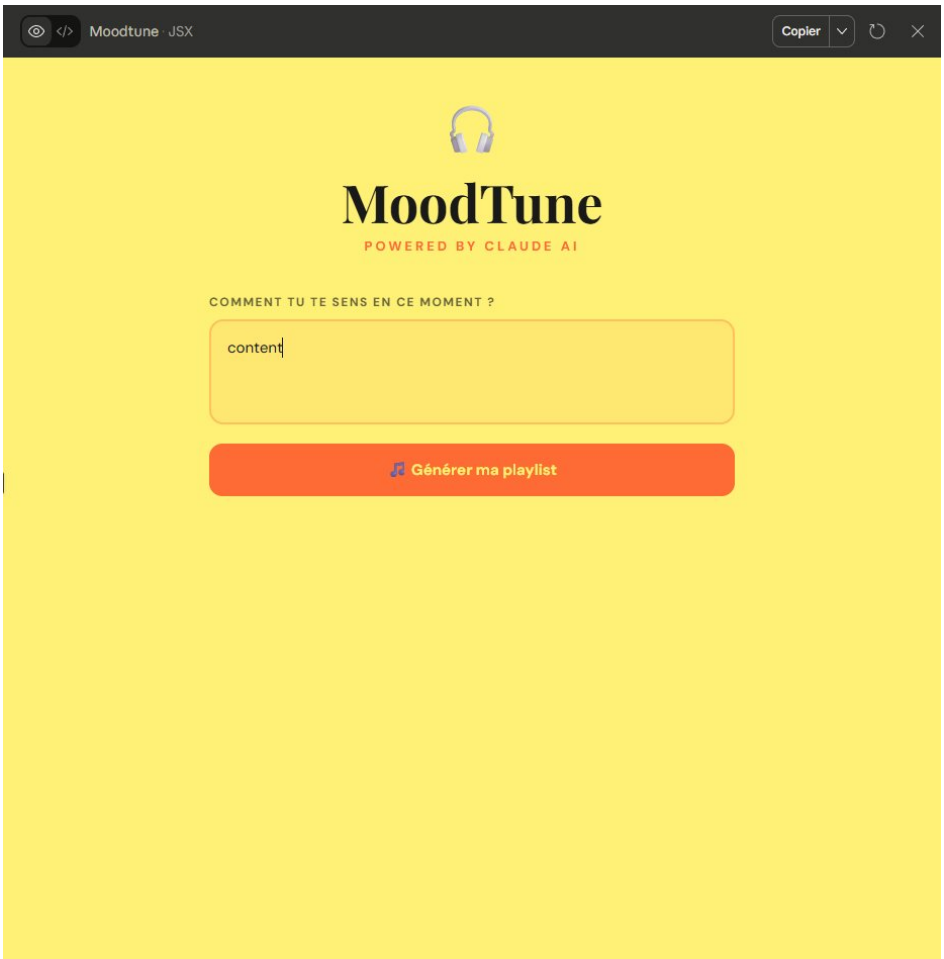
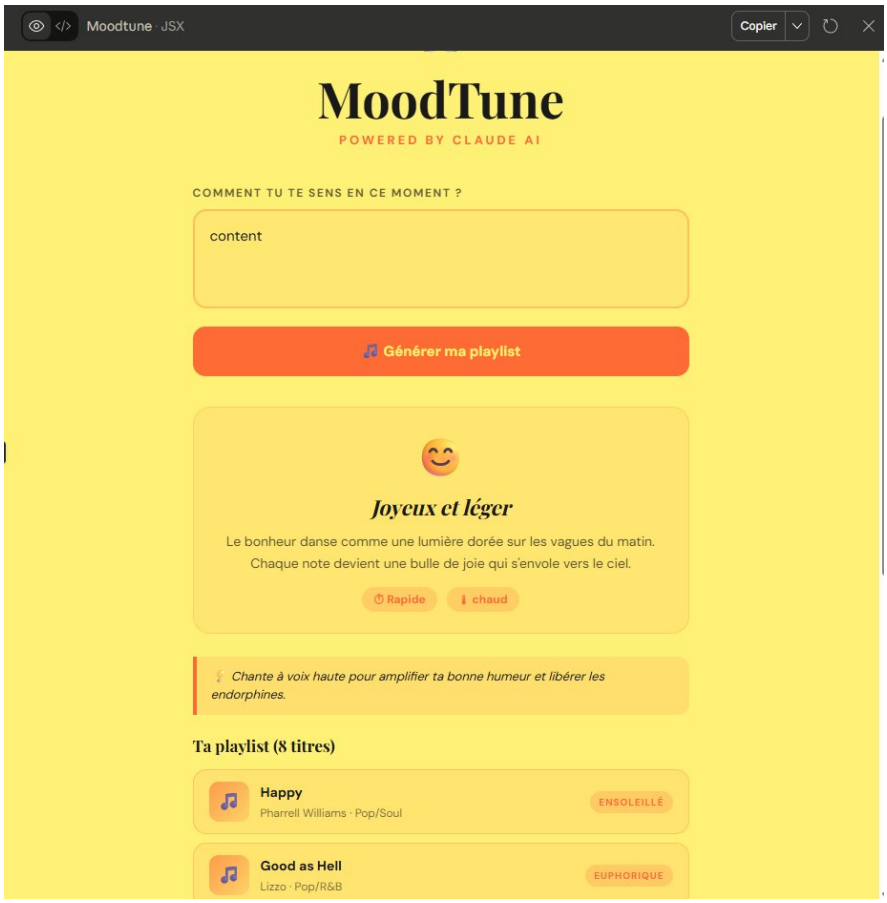


Figure 2: Saisie de lḿumeur – le thème change dynamiquement en temps réel

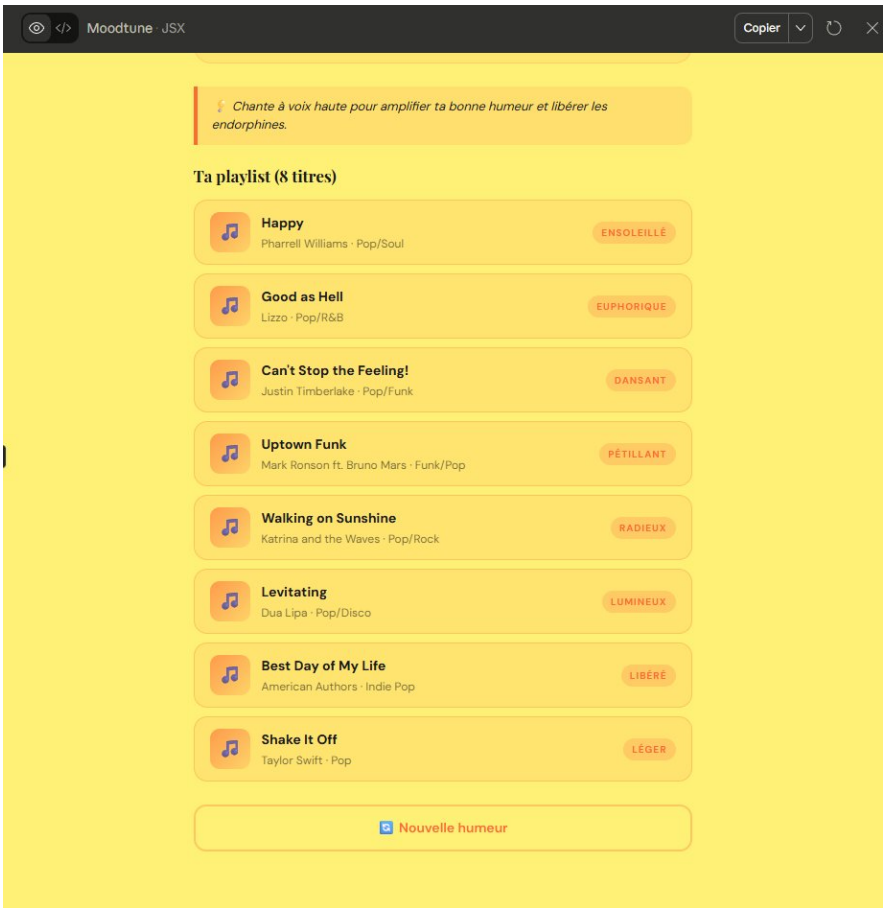
Étape 3 – Résultat : carte dḿumeur et début de playlist

Après génération, lÍA affiche le label *"Joyeux et léger"*, une description poétique, le tempo (Rapide), la couleur émotionnelle (chaud) et un conseil bien-être. Les premiers titres apparaissent en cascade.



Étape 4 – Playlist complète

Les 8 titres sáffichent avec leur artiste, genre et ambiance. Un bouton *"Nouvelle humeur"* permet de recommencer.



Annexe C: Manuel utilisateur

Prérequis

- Un navigateur web moderne (Chrome, Firefox, Edge)
- Une connexion Internet

Utilisation

1. Ouvrir l'application dans le navigateur
2. Décrire son humeur dans le champ de texte (ex : *"je suis fatigué mais motivé"*)
3. Appuyer sur **Entrée** ou cliquer sur *"Générer ma playlist"*
4. Patienter quelques secondes pendant la génération
5. Consulter la playlist et le conseil bien-être
6. Cliquer sur *"Nouvelle humeur"* pour recommencer

Remarques

- Plus la description est précise, plus la playlist sera pertinente
- L'interface change de couleur automatiquement selon les mots-clés détectés

- Les titres musicaux sont des suggestions de l'IA, non liées à un service de streaming